

졸업 프로젝트 10월 1차 격주보고서		
SNN을 활용한 고효율 추천 시스템 개발 및 최적화 연구		
2020058259 이승현 2020061676 이강혁		지도교수 김상욱 교수님
지난 2주간 진행사항	✓ 테스트 성능 판단 및 피드백	O
	✓ 시연	X
진행 내용	1. SpikingJelly 사용 ebrains 사이트에서 원격으로 SpiNNaker를 사용할 수 있도록 기능을 제공하여서 이를 이용하여 snn을 구현하고자 하였습니다. 하지만 저희가 생각지 못했던 문제가 발생하여 SpiNNaker 사용을 보류하였습니다. 아래는 그 이유입니다. (1) 통신오버헤드 저희는 ann도 결국 서버 통신을 통해 정보를 주고 받는 것이기에 snn 또한 다르지 않을 것이라 생각했습니다. 하지만 저희의 생각과는 다르게 차이가 있었습니다. ANN의 경우 보통 batch 단위로 대용량 데이터를 한 번에 보내는 것이 가능합니다. 이에 반해, SpiNNaker 칩은 이러한 기능을 제공하지 않으며 스파이크를 패킷 단위로 주고 받습니다. 이는 뇌 시뮬레이션 환경에서는 적합하지만, 추천 시스템에서의 데이터를 전송 할 때 적합하지 않고, 이 때문에 지연이 생기게 됩니다. 이러한 대용량 데이터를 다루는 경우, IBM TrueNorth나 Intel Loihi 같은 뉴로모픽 칩이 조금 더 적합하지만, 이 칩들은 현재 구매나 대여가 불가능한 상태입니다. 또한, SNN의 시간 정보를 다루는 특성으로 인해 time steps라는 하이퍼 파라미터를 갖게 됩니다. 이는 뉴런이 임계치를 초과하는지 판단 하기 위해 스파이크를 어느 정도의 시간 동안 모을 것 인지를 정하는 하이퍼 파라미터입니다. 다만, 이는 어느정도 이론적이며, 실제로는 spike를 발화하고 다음 발화가 가능할 때까지의 지연 시간 t를 설정하여 time step을 t로 나눈 값만큼 루프를 돌게 됩니다. 이때 이 루프마다 통신 지연이 발생하게 됩니다. 이러한 지연은 칩의 실행 시간을 늘리며 칩의 에너지 측정 시간이 늘어나 정확한 에너지 소모량을 측정할 수 없게 만듭니다.	

(2) quota 제한

ebrains에서는 다수의 원활한 사용 환경을 위해 프로젝트 별로 사용할 가능한 quota를 제한합니다. quota의 단위는 (core-hours)로 몇 개의 코어를 몇 시간 동안 사용했는지 측정합니다. 기본적으로 5000 quota가 제공되면 그 이상을 사용하고자 한다면, 별도의 승인을 받아야 합니다. 다만, 저희는 암시적 데이터를 활용한 SNN 추천 시스템을 연구하고 있고, 암시적 데이터의 유저와 아이템 수가 60만 정도입니다. 이 데이터를 다룰 때 추론 과정에서 많은 코어를 사용하게 됩니다. 추가로 승인을 받아서 quota를 할당 받았지만, 여전히 quota가 부족한 문제가 있었습니다.

이런 문제로 저희는 뉴로모픽 칩을 사용하는 대신 PyTorch를 기반으로 하는 SpikingJelly를 사용하게 되었습니다. SpikingJelly는 ANN to SNN의 빠른 구조 변환을 하게 해주면서 정보 손실을 최소화할 수 있고, 그러면서 SNN의 구조를 이용할 수 있기에 이를 선택하게 되었습니다.

2. 평가 방식

암시적 피드백 기반 모델의 경우, 사용자의 클릭·장바구니·구매 등의 행동 로그만이 주어지고 명시적 평점이 없기 때문에, 보지 않은 아이템을 모두 부정으로 가정할 수 없어 평가 전략이 중요합니다. 이에 따라 세 가지 대표적인 평가 방식이 사용됩니다.

(1) Full Catalog Ranking

Full catalog 방식은 각 사용자가 상호작용하지 않은 모든 아이템 전체에 대해 점수를 예측하고, 그중 실제로 상호작용한 아이템의 순위를 측정한다.

이 방식은 가장 정확한 평가 방법으로, 모델이 전체 아이템 공간에서 얼마나 잘 구분하는지를 직접 반영합니다.

하지만 대규모 암시적 데이터셋(ex Taobao, Beibei, MBD)에서는 아이템 수가 수백만에서 수천만 개에 달하기 때문에, 모든 사용자에게 대해 전 아이템을 스코어링하려면 막대한 메모리와 연산량이 요구됩니다. 따라서 현실적으로 메모리와 시간 제한 내에서 수행하기 어렵습니다.

(2) Sampled Evaluation

Sampled 평가는 각 사용자마다 실제로 상호작용한 아이템과 함께 무작위로 선택한 일부 negative sample 만을 후보로 두고 랭킹을 수행합니다.

예를 들어 한 사용자에게 대해 1 positive + 999 negative의 1000개

아이템만 평가하여 Hit@K, NDCG@K 등을 계산합니다.

이 방식은 계산량을 수천-수만 배 절감하면서도 상대적인 모델 간 성능 비교에는 유효합니다. 주로 대규모 로그 기반 추천 시스템 연구에서 개발·튜닝 단계의 효율적 비교를 위해 표준적으로 사용됩니다.

(3) Rerank Evaluation

Rerank 평가는 sampled 평가와 full-catalog 평가의 절충안으로, 모델이 candidate set을 생성한 뒤, 이 후보들을 다시 정밀하게 재정렬(reranking)하여 최종 순위를 평가하는 방식입니다.

예를 들어, 샘플링 단계에서 상위 1000개를 추출한 후, 그 내부에서 상대적 순위를 평가합니다.

이 방식은 full-catalog의 정확도와 sampled의 효율성을 절충하여 논문 보고용 평가나 SNN 변환 후 공정 비교에 적합합니다.

(4) 방대한 암시적 데이터셋에서 Sampled 평가를 사용하는 이유

암시적 피드백 데이터셋(특히 Taobao, Beibei, Alibaba MBD)은 수백만 명의 사용자와 수백만 개의 아이템, 그리고 수억 건 이상의 상호작용 로그를 포함합니다.

이 환경에서 full-catalog 방식으로 모든 사용자 x 모든 아이템의 예측 점수를 계산하면, 단일 평가조차 수백억 회의 내적 연산이 필요합니다.

반면 sampled 평가는 사용자당 일부 아이템만 선택하므로 평가 시간을 수 분에서 수십 분 수준으로 단축시킬 수 있어, 실제 연구나 논문에서는

- 모델 간 상대적 성능 비교에는 sampled 평가를,
- 최종 보고용 절대지표에는 rerank 혹은 full-catalog 평가를 사용하는 것이 일반적입니다.

결과적으로 sampled 평가는 방대한 암시적 데이터셋에서 현실적으로 효율적이며 공정한 모델 비교를 가능하게 합니다.

3. SNN 사용 알고리즘

(1) Time-to-First-Spike (TTFS) Coding

연속 데이터 값을 스파이크의 첫 발화 시간으로 부호화하여, 값이 클수록 더 빠르게 한 번만 발화하도록 합니다. rate coding과 달리 스파이크 수를 크게 줄일 수 있어 에너지·연산 측면에서 이점이 있습니다. 또한, TTFS는 Early Exit을 가능하게 만듭니다.

(2) Soft-reset Integrate-and-Fire (IF) Neuron Dynamics

임계치에서 발화 후 임계치를 초과한 막전위 값을 버리지 않고 보존한 채로 다음으로 넘기는 방식으로 동작합니다. 버려지는 정보량의 차이로 인해 발생할 수 있는 오차를 줄일 수 있습니다. 또한, 정보가 각 단계 사이에 연속적으로 전달되어 작은 timestep에서도 성능 저하를 완화할 수 있습니다.

(3) Spike-threshold Calibration

데이터 기반 통계로 뉴런의 발화 임계치를 보정합니다. 임계치가 낮으면 과발화, 높으면 과소발화로 이어지기 때문에, 적절한 값을 선택해 스파이크 발생률과 첫 발화 시점을 맞추어야 합니다. 이 보정은 t^* (평균 첫 발화 시간)를 줄이면서도 성능 유지하도록 합니다. 즉, 정확도와 에너지 효율을 향상시키는 알고리즘입니다.

(4) Temporal Early Exit by First-Spike

timestep을 진행하다가 상위 랭크 후보의 스파이크가 먼저 관측되면 그 시점에서 계산을 중단하는 Early Exit입니다. 이는 평균 추론 시간을 크게 줄일 수 있습니다. 대형 행렬곱을 한 번에 계산하는 ANN과 달리, 스파이크 단위로 계산하는 SNN에서 유용합니다. 시간 지연을 줄이는 알고리즘입니다.

4. ANN과 SNN 추론 결과 비교

다음은 대규모 암시적 데이터 추론에서 NCF와 SNN을 비교한 결과입니다.

```
Using device: cuda
[Auto Select Timesteps] candidates=[64, 128, 256], ANN Hit@10=0.204 -> selected Timesteps=64
NCF (ANN) Evaluation: 100% | 500/500 [02:26<00:00, 3.41it/s]
SNN Evaluation: 100% | 500/500 [03:33<00:00, 2.34it/s]
```

최종 성능 및 효율 비교

Metric	NCF (ANN)	SNN
Hit@10	0.2040	0.1920
NDCG@10	0.1172	0.1214
Latency (ms/user)	4.11	135.53
Ops (MACs/Synaptic)/user	6,246,240	64,064
Memory Access Count/user	409,604	192,192
ANN Reuse Factor (RF)	25	N/A
SNN Spike Rate (eff)	N/A	0.0235
[theoretical] Latency (ms/user)	N/A	42.49
[theoretical] TrueNorth user item-par (ms)	N/A	42.49
[theoretical] Loihi user item-par (ms)	N/A	10.62
[theoretical] SpiNNaker user item-par (ms)	N/A	42.49
[theoretical] Energy Score/user	21,736,915	516,356

Latency: user 한 명에 대한 아이템추론에 걸리는 실제 그래픽 카드 계산 지연

Ops: 1 user 당 ANN의 곱연산 수 / SNN의 평균 spike 발화 수.

Memory Access Count: 1 user 당 계산을 위해 메모리에 접근한 수

ANN Reuse Factor: 메모리 재사용성

SNN Spike Rate: 1 timestep (1 번의 전파)에서 임의의 시냅스에서 발화할 확률

[theoretical] Latency: 뉴로모픽 칩 사용 가정 시 이론으로 추측한 latency

[theoretical] Energy score: 위 지표들을 통해 이론적으로 추측한 에너지 사용량

결과를 보면 정확도에서 유의미한 차이가 나지 않고, 속도는 ANN이 우세, 에너지 효율성은 SNN이 우세한 것을 볼 수 있습니다.

에너지 효율성에서는 SNN이 0.023 정도의 발화율을 갖기 때문에 ANN에 비해 뛰어나지만, 반면에, latency에서는 칩 사용을 가정한 이론적인 지표를 적용하였음에도 ANN이 SNN에 비해 뛰어난 것을 알 수 있습니다.

이 결과를 통해 정확도 면에서 대규모 암시적 데이터의 매우 sparse한 특징이 sparse한 데이터에 강점이 있는 SNN과 잘 맞는다고 생각됩니다. 다만, 속도 면에서는 부족한 부분이 있어, 이는 에너지 효율성과의 트레이드 오프로 봐야 할 것 같습니다.

	다만, 추론의 정확도를 높이기 위해 사용되는 최적화 알고리즘으로 인해 실제 실행 시간의 비율이 추론 latency에 비해 크지 않다는 점과 뉴로모픽 칩에 대한 연구가 아직 진행 중에 있다는 점을 통해 SNN을 추천 시스템에 적용하는 것이 앞으로 분명 이점이 있을 것이라 생각합니다.
진행 중 이슈	
다음 2주간 계획	코드 리팩토링 및 taobao 외 다른 데이터셋 적용 후 시연